Code Review and Summary Report

Overview

This workspace contains two Java-based test automation projects:

- PactContracts_UI_Regression: contract-testing focused tests using Pact and WireMock.

- Selenid_ProductPage: UI automation tests using Selenide and JUnit for a web storefront.

1. Project 1: PactContracts_UI_Regression

Purpose

This module validates API contracts between a consumer application named storefront-web and a provider service named catalog-service.

Main Technologies

- Java 22

- Maven- JUnit 5

- Pact (consumer and provider)

- WireMock

- RestAssured

- Selenium/Selenide

- Testcontainers

What the code does

The project defines consumer-side Pact tests that describe expected HTTP behavior for the catalog service. These tests simulate requests and responses for:

- fetching an order by SKU

- creating a new order

- handling a non-existent order

- handling a temporary service outage

Key files

- PactContracts_UI_Regression/src/test/java/ust/gama/sdet/Gate4/PactContract/contracts/store/StoreFrontWebPactTest.java

Defines Pact contracts for successful and negative scenarios.

- PactContracts_UI_Regression/src/test/java/ust/gama/sdet/Gate4/PactContract/contracts/store/StoreFrontWebNegativePactTest.java

Covers the 404 scenario for a missing order.

- PactContracts_UI_Regression/src/test/java/ust/gama/sdet/Gate4/PactContract/contracts/catalog/catalogServiceVerification.java

Acts as the provider verification test using WireMock to stub the catalog service responses.

- PactContracts_UI_Regression/src/test/java/ust/gama/sdet/Gate4/PactContract/contracts/store/StoreFront.java

Implements the client-side calls to the catalog endpoints using RestAssured.

Observed behavior

- The consumer expects a GET endpoint at /catalog/sku/{id} to return order details.

- It also expects a POST endpoint at /catalog/orders to create an order.

- It validates error handling for missing orders and service unavailability.

- The provider verification uses WireMock stubs to simulate the service responses.

2. Project 2: Selenid_ProductPage

Purpose

This module automates UI checks for a product page and cart behavior in a web application.

Main Technologies

- Java 21/22

- Maven

- JUnit 5

- Selenide

- Selenium

What the code does

The test suite opens a product page, verifies that the selected product is in stock, adds it to the cart, and confirms the cart badge updates to 1.

Key files

- Selenid_ProductPage/src/test/java/ust/gama/sdet/Gate4/selenide/tests/AvailabiltyBadgeTest.java

  Contains the main UI tests.

- Selenid_ProductPage/src/test/java/ust/gama/sdet/Gate4/selenide/helper/Config.java

  Provides default base URLs and environment-based configuration helpers.

- Selenid_ProductPage/src/test/java/ust/gama/sdet/Gate4/selenide/helper/TestConfig.java

  Applies browser settings such as base URL, browser type, size, and headless mode.

- Selenid_ProductPage/src/test/java/ust/gama/sdet/Gate4/selenide/pages/ProductPage.java

  Encapsulates page interactions such as opening the product page, checking stock, and adding to cart.

-
Selenid_ProductPage/src/test/java/ust/gama/sdet/Gate4/selenide/pages/CartPage.java

Verifies the cart count badge.

Observed behavior

- The tests assume the application is running locally at http://localhost:5173.

- The product page uses data-test selectors such as detail-name, availability-badge, add-to-cart, and cart-count.

- The test flow is:

1. Open the product page for running-shoes.
2. Confirm the stock badge shows IN STOCK.
3. Add the item to the cart.
4. Confirm the cart count becomes 1.

3. Overall Summary

The workspace demonstrates two complementary automation strategies:

- Contract testing with Pact for validating service interactions.

- UI automation with Selenide for verifying end-user behavior in a browser.

Together, these projects cover both backend contract expectations and frontend user flows, making the workspace suitable for regression and integration testing.

4. Notes

- The Selenide tests appear to depend on a locally running web app.

- The Pact tests depend on a local Pact Broker and WireMock stubs for provider verification.

- Some test names and class names contain minor spelling inconsistencies, such as AvailabiltyBadgeTest instead of AvailabilityBadgeTest.